Evidence-Linked Clinical Trial Screening with a Compact Autoregressive Transformer

Muhammad Abdullah Masood
I230756

Abstract—A six-layer CPU decoder was adapted to match clinical trial criteria to patient notes. Across 206 held-out annotations, accuracy rose from 22.8% to 61.2%, but the adapted model predicted just two categories and retrieved no evidence. This divergence makes accuracy alone misleading. Three decoding implementations returned identical tokens across three controlled workloads. A local browser application exposes the decisions, evidence gaps and runtime differences.

Index Terms—trial screening, evidence retrieval, autoregressive decoding

I. Introduction

A recorded age can resolve an age threshold, but an unmentioned medication cannot resolve a contraindication. We ask whether an open CPU decoder can jointly generate a criterion judgment and the sentence IDs that support it. Trial-level summaries aggregate individual predictions by explicit rules. The annotations supply ground truth at criterion level.

II. Related Work and Architecture

TrialGPT separates trial retrieval, matching and ranking [1]. Supplying criteria directly isolates its matching task here. We compare against the same decoder before adaptation because the published TrialGPT results use another protocol. The Pythia suite supports controlled architectural investigation [2]. Our two candidates are Apache-2.0 GPT-NeoX decoders with byte-level BPE and a 2,048-position context [3], [4]. Pythia-14M has six layers of width 128 and four heads. Pythia-70M has six wider layers of width 512 and eight heads. Nominal FP32 weight storage is roughly 56 versus 280 MB before gradients, optimizer state and activations. The smaller model was selected for repeated CPU runs. Its performance relative to 70M remains unmeasured. LoRA reduces training state through low-rank updates [5]. Freezing the input embedding and selecting the output vocabulary here leaves about 1.20M trainable weights, allowing direct updates. FlashAttention tiles exact attention to reduce data movement [6]; this study tests its underlying recurrence independently. Speculative decoding amortizes target passes over draft proposals [7]. Short structured outputs restrict opportunities to recover drafting overhead.

III. Materials and Methods

A. Patient-level split and structured prediction

The NCBI benchmark [8] contains 1,015 rows from 53 patients. Lexically sorted unique patient IDs were shuffled with NumPy default_rng(42). The prescribed 37/8/8 patient groups contain 603/206/206 annotations. A separate check reconstructed the split, checked disjointness and accounted for every annotation. Only patient note, criterion type and criterion text enter the prompt. Expert labels and original sentence IDs are targets. GPT-generated columns and the legacy split flag are excluded. A missing training criterion is preserved with an explicit missing-text marker. Figure 1 distinguishes prediction from verbatim sentence lookup. The decoder emits a label symbol, ascending unique sentence IDs and a stop symbol. Type and available sentence numbers constrain legal outputs. The 107-row head is initialized from corresponding pretrained output rows, avoiding a projection onto all 50,304 tokens. Before and after adaptation the same output grammar applies. The deployed network contains 7,642,496 parameters. Classification explanations describe the model judgment and indicate missing support.

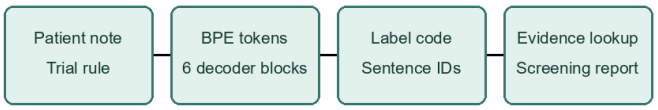


Fig. 1. Screening workflow from original text to reviewable output.

B. Training and model selection

Training used four FP32 epochs on two CPU threads, Python and PyTorch seed 42, teacher-forced answer-only cross-entropy, AdamW at 0.0002, weight decay 0.01, accumulation over four records and gradient norm clipping at 1. The final partial batch also takes a step. The input embedding stays frozen. Prompt lengths peaked at 453/322/361 tokens across train/validation/test. No benchmark note was truncated; the app rejects inputs above 1,930 prompt tokens. The highest validation six-class macro-F1 selects the checkpoint, with earliest epoch winning ties. Its hash was recorded before final test scoring. Figure 2 shows decreasing loss and a much smaller validation gain after epoch one. Epoch four reached 0.219 validation macro-F1.

C. Decoding and attention

Uncached inference recomputes the prefix. Cached inference stores rotated keys and values for each layer. The

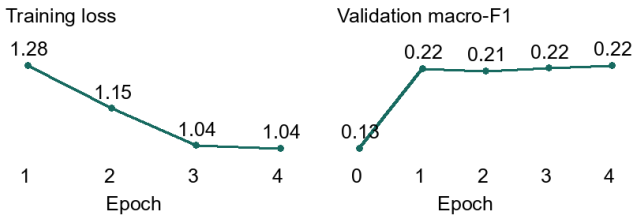


Fig. 2. Training loss and validation macro-F1. Epoch zero denotes the base decoder.

next token position equals the cached length, and the mask permits keys through the absolute query position. Shape-dependent rounding can affect a near-tied argmax. The draft uses the first two pretrained blocks and the same tokenizer and restricted head vocabulary. It proposes up to four tokens. One target pass verifies a block. On rejection both caches are sliced to the accepted prefix and the corrected target token remains pending. Full agreement permits an extra target token. Reference attention applies stable row softmax to scaled query-key dot products after masking. Each query tile instead maintains maximum m, denominator z and numerator a. Given scores S, let $m' = \max(m, \max S)$ and $P = \exp(S - m')$. Update $z' = e^{m-m'}z + \sum P$ and $a' = e^{m-m'}a + PV$, then divide the final a by z. Blocks have width 32. Wholly future blocks are skipped; partial blocks use absolute causal offsets. This educational kernel is tested separately. The deployed decoder uses PyTorch scaled dot-product attention.

IV. Experimental Setup and Metrics

Experiments used an HP EliteBook 840 G8 with 15.7 GiB RAM and CPU-only PyTorch 2.8.0. The validation reference audit compared restricted logits against the official Transformers implementation at 5, 32 and 96 positions. All 133 argmax decisions agreed. Maximum absolute logit difference was 0.00246; the custom path uses a different residual addition order and attention implementation. Cached versus full-prefix chunk checks had a maximum absolute difference of 0.000059 and identical argmax tokens. Accuracy and macro-F1 use fixed class sets (six overall and four by type). Undefined class scores are zero. Evidence micro scores pool overlaps and set sizes; macro scores average per-record precision, recall and F1. Two empty sets receive one for each per-record score. A one-sided empty denominator receives zero. Benchmark prompts use validation prefixes and controlled fixed output budgets, with stop disabled. Each CPU timing uses a warm-up and median of five synchronous repetitions. Sampled process RSS includes the interpreter, both models and allocator. It can miss short-lived peaks. Prefill and decode medians are timed separately.

V. Results and Discussion

Table I shows that supervised adaptation improves classification on both splits while eliminating positive evidence retrieval. The high macro evidence score arises from correctly empty sets. Table II shows stronger exclusion accuracy. The model frequently predicts not excluded in this group. Table III shows that all correctly classified cases come from two labels. Excluded has no test support. Full per-type tables and confusion matrices are included as machine-readable metrics. Table IV locates the largest systematic mistakes. Every included and not-included example is assigned insufficient information. Table V shows token agreement in all cases (logs include the assertions). Caching helps most on the longest prefix. Draft acceptance was 1/7, 2/22 and 7/39. Verification overhead made speculation slower than cached greedy at all three lengths. Table VI shows FP32 agreement across causal and unmasked shapes. Python block dispatch dominates at these lengths. The score tile has at most 32 by 32 entries per head; process RSS includes retained buffers and does not directly measure this saving.

TABLE I
Paired classification and evidence scores

Split	Model	Acc.	Macro-F1	Ev. micro	Ev. macro
Validation	base	0.189	0.127	0.116	0.524
Validation	adapted	0.621	0.219	0.000	0.563
Test	base	0.228	0.145	0.144	0.590
Test	adapted	0.612	0.222	0.000	0.704

TABLE II
Adapted test classification by criterion type

Group	Rows	Accuracy	Macro-F1
all	206	0.612	0.222
inclusion	75	0.507	0.168
exclusion	131	0.672	0.202

TABLE III
Adapted test performance by expert category

Class	Support	Precision	Recall	F1
included	27	0.000	0.000	0.000
not included	7	0.000	0.000	0.000
excluded	0	0.000	0.000	0.000
not excluded	91	0.693	0.967	0.807
not enough information	66	0.481	0.576	0.524
not applicable	15	0.000	0.000	0.000

TABLE IV
Adapted test confusion matrix; class order follows Table III

Gold / pred.	A	B	C	D	E	F
A	0	0	0	0	27	0
B	0	0	0	0	7	0
C	0	0	0	0	0	0
D	0	0	0	88	3	0
E	0	0	0	28	38	0
F	0	0	0	11	4	0

TABLE V
Generation timing in milliseconds and peak process RSS

In/out	Path	Prefill	Decode	Tok/s	MiB
32/4	uncached	7.7	22.8	131.2	333
32/4	cached	6.2	16.2	178.7	333
32/4	speculative	–	–	78.5	334
96/8	uncached	11.9	90.6	78.0	341
96/8	cached	12.2	42.3	146.9	341
96/8	speculative	–	–	60.9	338
192/16	uncached	28.1	397.4	37.6	347
192/16	cached	21.7	109.1	122.4	347
192/16	speculative	–	–	79.4	347

TABLE VI
Attention agreement and CPU runtime (milliseconds)

Length	Causal	Max err.	Mean err.	Full	Tiled
17	yes	3.6e-07	2.8e-08	0.19	0.79
17	no	3.6e-07	3.4e-08	0.35	0.48
65	yes	4.8e-07	3.0e-08	0.47	3.73
65	no	3.6e-07	2.7e-08	0.35	3.48
256	yes	3.6e-07	2.7e-08	2.60	15.85
256	no	3.6e-07	2.4e-08	1.13	17.50

A. Eight held-out errors

Row 159 misses the explicitly recorded age of 67 and its citation. Row 161 assigns uncertainty to an elective carotid procedure experts marked not included; the note describes cardiac catheterization. Row 165 changes a not-applicable pregnancy criterion into not excluded. Row 169 clears a low-filtration exclusion without the numerical evidence required to resolve it. Row 179 predicts the expert catheterization label but omits sentence 0. Row 306 correctly signals uncertainty about liver resection but omits sentence 3 describing drainage of an abscess. Row 322 loses sex-related context in sentence 0 and changes the expert exclusion judgment to uncertainty. Row 422 misses the birth-control applicability judgment and sentence 3. Frequent labels and early termination after a label are plausible contributors; their causal roles remain untested.

B. Browser application

Figure 3 shows the local workflow. Comparison mode runs all three decoders on the entered criteria. Text export includes predictions, evidence and provenance. A cited failed inclusion or positive exclusion flags a potential conflict. Uncertainty or missing citations trigger review. Other cases report no flagged conflict among entered criteria. No trial-level outcome is treated as a benchmark label.

VI. Limitations and Conclusion

Only eight patients contribute to the test metrics. Repeated criteria from a patient are correlated, and one class has zero support. Accuracy gains conceal loss of evidence retrieval. This model cannot support clinical enrollment decisions. Its reproducible contribution is a small experimental system that exposes these failures and tests whether alternative inference paths preserve the same outputs.

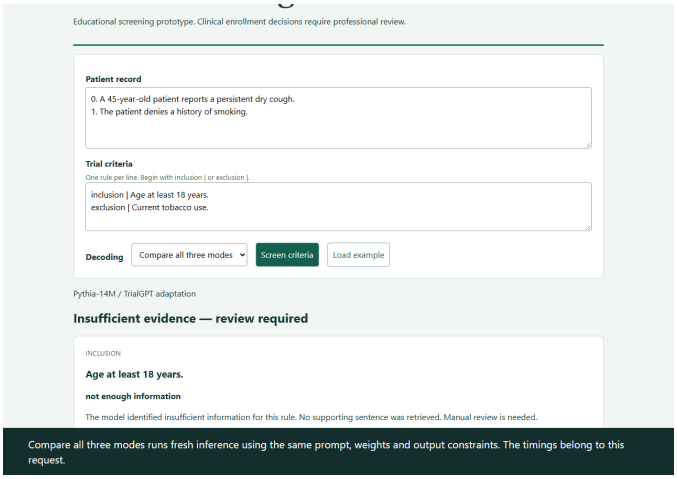


Fig. 3. Browser interface during the recorded local demonstration.

AI-use appendix

OpenAI Codex assisted with planning, code, experiment execution, error analysis, report drafting and the captioned browser demonstration. The submission contains the measured outputs and checkpoint. The student is responsible for understanding the implementation and conducting the live defense.

References

[1] Q. Jin et al., Matching patients to clinical trials with large language models, Nature Communications, 2024, doi:10.1038/s41467-024-53081-z.
[2] S. Biderman et al., Pythia: A suite for analyzing large language models across training and scaling, ICML, 2023.
[3] EleutherAI, Pythia-14M model card and configuration. https://huggingface.co/EleutherAI/pythia-14m
[4] EleutherAI, Pythia-70M model card and configuration. https://huggingface.co/EleutherAI/pythia-70m
[5] E. Hu et al., LoRA: Low-rank adaptation of large language models, ICLR, 2022.
[6] T. Dao et al., FlashAttention: Fast and memory-efficient exact attention with IO-awareness, NeurIPS, 2022.
[7] Y. Leviathan, M. Kalman and Y. Matias, Fast inference from transformers via speculative decoding, ICML, 2023.
[8] NCBI, TrialGPT criterion annotations. https://huggingface.co/datasets/ncbi/TrialGPT-Criterion-Annotations